

# Semana 02 – Geração de Código Intermediário

**Disciplina:** Compiladores

**Curso:** Ciência da Computação

**Docente:** Prof.<sup>a</sup> Ma. Layse Souza

**Grupo:** 05

## 1. Objetivo

O objetivo principal desta etapa foi desenvolver o módulo de tradução das regras sintáticas da linguagem RoboLang para uma **Representação Intermediária (IR)**.

Nesta semana, o foco consistiu em:

- Demonstrar a conversão das instruções de alto nível para uma estrutura estruturada e portátil (formato JSON).
- Implementar o mecanismo de varredura da árvore sintática utilizando o padrão *Listener*.
- Validar a solução através de cenários de teste que comprovem a correta hierarquização dos comandos.

## 2. Estrutura Implementada

Para realizar a tradução da linguagem fonte para a Representação Intermediária, a implementação baseou-se nas ferramentas geradas pelo ANTLR (reconhecidas nos arquivos do projeto) e na lógica de programação em Python. A arquitetura foi dividida da seguinte forma:

- Gramática e Analisadores (Lexer/Parser):  
A base da estrutura continua sendo a definição formal da linguagem, que estabelece os tokens (como MOVER, VIRAR, REPETIR) e as regras gramaticais. O analisador sintático gera uma árvore de análise (Parse Tree) que serve de entrada para o tradutor.
- Padrão Listener (Ouvinte):  
A estratégia adotada para a geração da IR foi o uso da classe RoboLangListener. Diferente de percorrer a árvore manualmente, o sistema utiliza um mecanismo de eventos. Cada vez que o analisador entra ou sai de uma regra gramatical (como enterMover ou exitRepetir), um método específico é acionado.
- Construtor da IR:  
Foi desenvolvida uma lógica interna que, ao capturar esses eventos do Listener, popula uma estrutura de dados em memória (listas e dicionários). Ao final da leitura do arquivo de entrada, essa estrutura é serializada e exportada para o formato JSON. Essa escolha garante que o código intermediário seja leve, legível por humanos e facilmente consumido pelas próximas etapas do compilador.

### 3. Processo de Tradução

A tradução ocorre mapeando cada comando da RoboLang para um objeto JSON com atributos específicos. O processo segue as seguintes definições:

1. **Comandos de Ação (Mover/Virar):** São traduzidos para objetos contendo o tipo da instrução (ex: "mover"), o parâmetro principal (eixo ou direção) e a magnitude (valor numérico e unidade).
2. **Comandos de Configuração (Velocidade/Esperar):** São convertidos em objetos que registram a alteração de estado do robô ou pausas temporais.
3. **Estruturas de Controle (Repetir):** Este é o ponto mais complexo da tradução. O comando `repetir` é mapeado como um objeto que contém uma propriedade "vezes" (número de iterações) e uma lista interna de "comandos", criando uma estrutura aninhada (hierárquica) na saída JSON.

### 4. Validação e Exemplos (Cenários de Teste)

Para validar a geração da Representação Intermediária, foram utilizados três exemplos distintos, cobrindo comandos simples, parametrização e estruturas de repetição.

#### Cenário 1: Movimentação Simples

Entrada: Um comando para mover o robô no eixo X por uma distância específica.

Resultado (IR Gerada): O sistema gerou um objeto JSON identificando a ação "mover", especificando o eixo "x", o valor "10" e a unidade "cm". A estrutura foi identificada corretamente sem erros de sintaxe.

#### Cenário 2: Configuração de Estado

Entrada: Comandos para ajustar a velocidade e realizar uma rotação.

Resultado (IR Gerada): Foi gerada uma lista contendo dois objetos sequenciais. O primeiro define a "velocidade" com seu valor numérico. O segundo define a ação "virar" com o ângulo e a unidade "graus". A ordem sequencial foi preservada fielmente na saída.

#### Cenário 3: Estrutura de Repetição (Loop)

Entrada: Um comando de repetição contendo instruções internas (escopo aninhado).

Resultado (IR Gerada): O tradutor produziu um objeto do tipo "repetir". Dentro deste objeto, foi criado um campo "corpo" (body) contendo a lista de instruções que devem ser repetidas. Isso valida a capacidade do compilador de lidar com escopos e blocos de código, representando corretamente a hierarquia da árvore sintática no formato JSON.

## 5. Conclusão

A segunda etapa do projeto foi concluída com êxito. A utilização do padrão *Listener* fornecido pelo ANTLR permitiu desacoplar a lógica de tradução da lógica de análise sintática, resultando em um código mais limpo e modular.

A Representação Intermediária em JSON gerada reflete com precisão a estrutura do código fonte original, validando os requisitos de tradução e hierarquização. Com a IR pronta e validada, o projeto está preparado para avançar para as etapas de otimização ou interpretação final dos comandos.